

SHA-256 Operation Explained

1. OVERVIEW OF SHA-256

SHA-256 (Secure Hash Algorithm 256-bit) is a cryptographic hash function in the SHA-2 family. It transforms input data of arbitrary size into a fixed 256-bit (64 hexadecimal character) output. Unlike MD5, SHA-256 remains cryptographically secure and collision-resistant.

Security Standard: SHA-256 is a one-way function with pre-image resistance (infeasible to reverse hash to input) and collision resistance (infeasible to find two inputs that produce the same output).

2. FUNDAMENTAL BITWISE OPERATIONS GUIDE

SHA-256 heavily relies on fundamental bitwise operations performed on 32-bit words:

AND (&) - 1 if both are 1

```
  0 1 1 1   (7)
& 0 0 0 1   (1)
-----
  0 0 0 1   (1)
```

OR (|) - 1 if at least one is 1

```
  0 1 1 1   (7)
| 0 0 0 1   (1)
-----
  0 1 1 1   (7)
```

XOR (^) - 1 if different (no carry)

```
  0 1 1 1   (7)
^ 0 0 0 1   (1)
-----
  0 1 1 0   (6)
```

NOT (~) - Inverts all bits

```
~ 0 1 1 1   (7)
-----
  1 0 0 0   (8 unsigned)
```

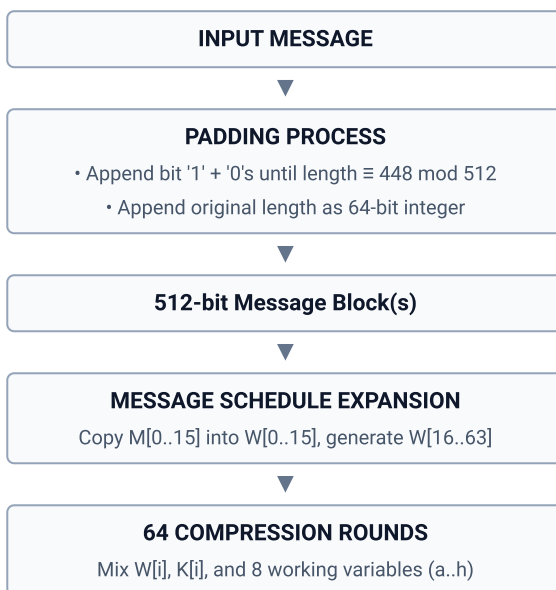
SHIFT RIGHT (>>) - Fills left with 0

```
  0 1 1 1   (7) >> 1
-----
  0 0 1 1   (3)
```

ROTATE RIGHT (ROTR) - Wraps to left

```
  0 0 1 1   (3) ROTR 1
-----
  1 0 0 1   (9)
```

3. HIGH-LEVEL DATA FLOW



4. STEP-BY-STEP OPERATION BREAKDOWN

1. Message Padding: The input is padded with a single 1 bit, followed by 0 bits until the bit length is $448 \bmod 512$. A 64-bit big-endian integer representing the original message length is then appended, producing 512-bit blocks.

2. State Initialization: SHA-256 initializes eight 32-bit state variables (H0 through H7) from the fractional parts of the square roots of the first 8 primes:

State	Initial Value (Hex)	State	Initial Value (Hex)
H0	0x6a09e667	H4	0x510e527f
H1	0xbb67ae85	H5	0x9b05688c
H2	0x3c6ef372	H6	0x1f83d9ab
H3	0xa54ff53a	H7	0x5be0cd19

5. MESSAGE SCHEDULE EXPANSION (CORRECTED NIST FORMULAS)

Each 512-bit block provides 16 32-bit words $M[0] \dots M[15]$. These are expanded into 64 words $W[0] \dots W[63]$. Words 0 to 15 are copied directly, and 48 additional words are generated using bitwise rotation and shift operations:

```
For i = 0 to 15: W[i] = M[i]
For i = 16 to 63: // Corrected NIST FIPS 180-4 standard
constants:  $\sigma_0(x) = \text{ROTR}(7, x) \text{ XOR } \text{ROTR}(18, x) \text{ XOR } (x \gg 3)$ 
 $\sigma_1(x) = \text{ROTR}(17, x) \text{ XOR } \text{ROTR}(19, x) \text{ XOR } (x \gg 10)$ 
W[i] = W[i-16] +  $\sigma_0(W[i-15])$  + W[i-7] +  $\sigma_1(W[i-2])$  (modulo  $2^{32}$ )
```

6. COMPRESSION FUNCTION & STATE UPDATE

Eight working variables a, b, c, d, e, f, g, h are initialized to the current state $H0 \dots H7$. Each block undergoes 64 rounds of mixing:

```
Ch(e, f, g) = (e AND f) XOR ((NOT e) AND g)
Maj(a, b, c) = (a AND b) XOR (a AND c) XOR (b AND c)
 $\Sigma_0(a) = \text{ROTR}(2, a) \text{ XOR } \text{ROTR}(13, a) \text{ XOR } \text{ROTR}(22, a)$ 
 $\Sigma_1(e) = \text{ROTR}(6, e) \text{ XOR } \text{ROTR}(11, e) \text{ XOR } \text{ROTR}(25, e)$ 
For i = 0 to 63:
T1 = h +  $\Sigma_1(e)$  + Ch(e, f, g) + K[i] + W[i]
T2 =  $\Sigma_0(a)$  + Maj(a, b, c)
h = g
g = f
f = e
e = d + T1 (modulo  $2^{32}$ )
d = c
c = b
b = a
a = T1 + T2 (modulo  $2^{32}$ )
```

Final Addition: After 64 rounds, the variables are added back to the running state ($H0 += a, H1 += b$, etc.). The final 256-bit hash is formed by concatenating $H0$ through $H7$.